Correttezza

Indice

Correttezza di un algoritmo
- Invarianti di ciclo

Correttezza di Find Maximum

Correttezza di Insertion Sort
- Il ciclo interno
- Il ciclo esterno

Correttezza di Binary Search

Correttezza dell'algoritmo di Euclide

Correttezza degli algoritmi ricorsivi

Correttezza di un algoritmo

Introduzione: Correttezza di un algoritmo

- In questa sezione definiamo con precisione quando un algoritmo è **corretto**
- Prima di chiedersi quanto sia veloce un algoritmo, dobbiamo essere sicuri che faccia ciò che promette
- Introduciamo lo strumento standard per dimostrare la correttezza di un ciclo: l'**invariante di ciclo**, con le sue tre proprietà (inizializzazione, conservazione, terminazione)

Prima della velocità, la correttezza

- I capitoli precedenti hanno misurato quanto costa un algoritmo: la dimensione del suo input e il suo tempo di esecuzione
- Prima di chiedersi quanto sia veloce un algoritmo, però, dobbiamo essere sicuri che faccia ciò che promette
- Questo capitolo presenta lo strumento standard per dimostrarlo, l'**invariante di ciclo**, e lo applica ad algoritmi già incontrati nei capitoli «Ordinamento», «Ricerca lineare e ricerca binaria» e «Algoritmi numerici»:
  - Find Maximum, Insertion Sort, Binary Search e l'algoritmo di Euclide.

La correttezza va dimostrata

- La prima parte dell'analisi di un algoritmo consiste nel determinarne la **correttezza**, cioè nel dimostrare che calcola l'output corretto per ogni possibile istanza di input
- Provare un algoritmo su alcune istanze può mostrare che è **sbagliato** (basta un controesempio)
- Ma non può mai mostrare che è corretto: le istanze di input sono infinite
- La correttezza va **dimostrata**

Definizione 1: algoritmo corretto

*Un algoritmo è **corretto** se, per ogni istanza di input I del problema, **termina** dopo un numero finito di passi e **restituisce l'output corretto** per I.*

Correttezza dei cicli

- Gli algoritmi che consideriamo sono costituiti da sequenze di istruzioni e da cicli
- Le sequenze di istruzioni sono facili da analizzare, mentre un ciclo esegue le stesse istruzioni un numero di volte che dipende dall'input

La correttezza di un algoritmo si basa sulla correttezza dei suoi cicli.

La correttezza di un ciclo si dimostra per mezzo di un **invariante di ciclo**.

Definizione 2: invariante di ciclo

*Un **invariante di ciclo** è una proprietà delle variabili di un algoritmo che è vera ogni volta che viene valutata la condizione di un ciclo, cioè prima della prima iterazione del ciclo e dopo ogni iterazione del ciclo.*

Le tre proprietà da dimostrare

Per dimostrare la correttezza di un algoritmo per mezzo di un invariante di ciclo dobbiamo mostrare tre cose:

1. **Inizializzazione**: l'invariante di ciclo è vero prima della prima iterazione del ciclo.
2. **Conservazione**: se l'invariante di ciclo è vero prima di un'iterazione del ciclo, allora rimane vero alla fine dell'iterazione, o equivalentemente, è vero prima dell'iterazione successiva.
3. **Terminazione**: il ciclo termina dopo un numero finito di iterazioni. Quando il ciclo termina, l'invariante di ciclo, insieme al fatto che la condizione del ciclo è falsa, fornisce una proprietà utile che mostra che l'algoritmo è corretto.

Somiglianza con l'induzione matematica

- Notiamo la somiglianza con l'**induzione matematica**, dove per dimostrare che una proprietà vale è necessario dimostrare un caso base e un passo induttivo:
  - mostrare che l'invariante vale prima della prima iterazione corrisponde al **caso base**;
  - mostrare che l'invariante vale da un'iterazione alla successiva corrisponde al **passo induttivo**.

La proprietà di terminazione differisce dal consueto uso dell'induzione matematica,
in cui il passo induttivo viene applicato infinite volte:
qui l'"induzione" si ferma quando il ciclo termina.

Come dimostrare che un ciclo termina

- Per dimostrare che un ciclo termina, di solito si individua un **numero naturale**, per esempio

  $$n - i + 1 \quad \text{o la lunghezza di un sotto-array,}$$

  che:
  - diminuisce strettamente a ogni iterazione;
  - deve essere positivo perché il ciclo continui.

- Poiché un numero naturale non può diminuire all'infinito, il ciclo termina

Schema di una dimostrazione di correttezza con un invariante di ciclo

1. Enunciare l'invariante: una proprietà delle variabili, vera ogni volta che viene valutata la condizione del ciclo.
2. **Inizializzazione**: verificarlo con i valori assegnati prima del ciclo.
3. **Conservazione**: assumerlo insieme alla condizione del ciclo, eseguire il corpo una volta, e verificarlo con i nuovi valori.
4. **Terminazione**: trovare un numero naturale che diminuisce strettamente a ogni iterazione; poi combinare l'invariante con la negazione della condizione del ciclo per ottenere l'output corretto.

Riepilogo

Algoritmo corretto:

- per ogni istanza di input, termina e restituisce l'output corretto
- i test possono solo mostrare che un algoritmo è sbagliato: la correttezza va dimostrata

Invariante di ciclo:

- una proprietà delle variabili, vera ogni volta che viene valutata la condizione del ciclo
- tre proprietà: inizializzazione, conservazione, terminazione
- inizializzazione e conservazione sono il caso base e il passo induttivo di un'induzione che si ferma quando il ciclo termina

Correttezza di Find Maximum

Introduzione: Correttezza di Find Maximum

- In questa sezione applichiamo l'invariante di ciclo al ciclo più semplice: una singola scansione di un array
- Enunciamo l'invariante dell'algoritmo `FindMaximum`, lo verifichiamo su un esempio e dimostriamo le tre proprietà
- La dimostrazione è il modello di tutte le dimostrazioni del capitolo

Il problema

Dato un array $a[1..n]$ di n numeri interi, con $n \geq 1$,
consideriamo il problema di determinare
il valore massimo dell'array.

- Il primo esempio è il ciclo più semplice: una singola scansione dell'array
- Questo è lo pseudocodice dell'algoritmo `FindMaximum`

Algorithm: FindMaximum

Input : Un array $a[1..n]$ di n numeri interi
Output : Il valore massimo dell'array a

```
1  function FindMaximum(a, n)
2      max := a[1]
3      i := 2
4      while i ≤ n do
5          if a[i] > max then
6              max ← a[i]
7          end
8          i ← i + 1
9      end
10     return max
11 end
```

Esegui in Colab: Python · C++ · Java

Invariante di ciclo di FindMaximum

- L'**invariante di ciclo** del ciclo while di `FindMaximum` è:

Ogni volta che viene valutata la condizione alla riga 4,
la variabile *max* è uguale al valore massimo del sotto-array $a[1..i-1]$:

$$max = \max\{a[1], a[2], \ldots, a[i-1]\}.$$

Esempio

Consideriamo l'array $a = [9, 12, 3, 16, 19, 11, 12]$ con $n = 7$. La tabella mostra i valori di i e max ogni volta che viene valutata la condizione alla riga 4:

i	max	$a[1..i-1]$	$\max\{a[1], \ldots, a[i-1]\}$	
2	9	[9]	9	
3	12	[9, 12]	12	
4	12	[9, 12, 3]	12	
5	16	[9, 12, 3, 16]	16	
6	19	[9, 12, 3, 16, 19]	19	
7	19	[9, 12, 3, 16, 19, 11]	19	
8	19	[9, 12, 3, 16, 19, 11, 12]	19	(il ciclo termina)

A ogni valutazione l'invariante vale. Quando il ciclo termina $i = n + 1 = 8$, e l'algoritmo restituisce 19, il massimo dell'intero array.

Usando l'invariante di ciclo possiamo dimostrare la correttezza dell'algoritmo `FindMaximum`,
enunciata dalla seguente osservazione.

Osservazione 1

Dato un array $a[1..n]$ *di* $n \geq 1$ *numeri interi, l'algoritmo* `FindMaximum` *termina e restituisce*

$$\max\{a[1], a[2], \ldots, a[n]\}.$$

Dimostrazione (1/3)

Dimostriamo le tre proprietà dell'invariante di ciclo.

Inizializzazione.

- Prima della prima iterazione, le righe 2 e 3 pongono $max = a[1]$ e $i = 2$
- Il sotto-array $a[1..i-1] = a[1..1]$ è costituito dal solo elemento $a[1]$, il cui massimo è $a[1] = max$
- Quindi l'invariante vale

Dimostrazione (2/3)

Conservazione. Supponiamo che, prima di un'iterazione, $max = \max\{a[1], \ldots, a[i-1]\}$. Il corpo del ciclo confronta $a[i]$ con max (riga 5):

- se $a[i] > max$, allora la riga 6 pone $max \leftarrow a[i]$, e $a[i]$ è maggiore di tutti gli elementi di $a[1..i-1]$;
- se $a[i] \leq max$, allora max non cambia ed è maggiore o uguale a $a[i]$.

In entrambi i casi, alla fine del corpo

$$max = \max\{a[1], \ldots, a[i-1], a[i]\}.$$

La riga 8 incrementa i di 1, quindi, con il nuovo valore di i, si ha di nuovo $max = \max\{a[1], \ldots, a[i-1]\}$. Quindi l'invariante vale prima dell'iterazione successiva.

Dimostrazione (3/3)

Terminazione.

- La variabile i parte da $2 \leq n+1$ e aumenta di 1 a ogni iterazione
- Quindi il ciclo termina dopo esattamente $n-1$ iterazioni, quando per la prima volta $i = n+1$
- Sostituendo $i = n+1$ nell'invariante di ciclo otteniamo

$$max = \max\{a[1], a[2], \ldots, a[n]\},$$

che è il valore restituito alla riga 10

Quindi l'algoritmo è corretto. □

Riepilogo

Correttezza di Find Maximum:

- invariante: $max = \max\{a[1], \ldots, a[i-1]\}$ ogni volta che viene valutata la condizione $i \leq n$
- inizializzazione: $a[1..1]$ ha il solo elemento $a[1] = max$
- conservazione: il confronto di $a[i]$ con max estende il massimo ad $a[1..i]$

Terminazione:

- esattamente $n-1$ iterazioni, il ciclo si ferma con $i = n+1$
- con $i = n+1$ l'invariante fornisce il massimo dell'intero array

Correttezza di Insertion Sort

Introduzione: Correttezza di Insertion Sort

- In questa sezione dimostriamo la correttezza di un algoritmo con **due cicli annidati**
- La dimostrazione richiede due invarianti, uno per ciascun ciclo
- Dimostriamo prima che cosa fa una singola esecuzione del ciclo interno (un lemma), e poi usiamo questo risultato per il ciclo esterno

Il problema

Dato un array $a[1..n]$ di n numeri naturali, con $n \geq 1$, consideriamo il problema di riordinare gli elementi dell'array in modo non decrescente:

$$a[1] \leq a[2] \leq \cdots \leq a[n].$$

- Con due cicli annidati la dimostrazione richiede due invarianti, uno per ciascun ciclo
- Questo è lo pseudocodice dell'algoritmo `InsertionSort`

Algorithm: InsertionSort (1/2)

Input : Un array $a[1..n]$ di n numeri naturali
Output : L'array a ordinato in modo non decrescente

```
1 function InsertionSort(a, n)
2 |   i := 2
3 |   while i ≤ n do
4 |   |   key := a[i]                    // Elemento da inserire
5 |   |   j := i − 1
```

Algorithm: InsertionSort (2/2)

```
function InsertionSort(a, n) (continua)
    while i ≤ n (continua) do
        while j ≥ 1 and a[j] > key do
            a[j + 1] ← a[j]   // Sposta l'elemento a destra
            j ← j − 1
        end
        a[j + 1] ← key   // Inserisce key nella posizione
         corretta
        i ← i + 1
    end
end
```

Esegui in Colab: Python · C++ · Java

La condizione del ciclo interno

- La condizione alla riga 6 è valutata da sinistra a destra, e la seconda parte è valutata solo se la prima è vera
- Quando $j = 0$ la condizione è falsa e l'elemento inesistente $a[0]$ non viene mai acceduto
- La condizione C++ `i > -1 and a[i] > key` si comporta allo stesso modo

Piano della dimostrazione

- L'algoritmo contiene due cicli annidati
- Dimostriamo prima che cosa fa una singola esecuzione del ciclo while **interno** (righe 6–8)
- Poi usiamo questo risultato per dimostrare la correttezza del ciclo while **esterno** (righe 3–11)

Il ciclo interno: notazione

- Consideriamo un'iterazione del ciclo esterno, con un dato valore di $i \in \{2, 3, \ldots, n\}$
- Indichiamo con

  $$b[1..i]$$

  il contenuto del sotto-array $a[1..i]$ all'inizio di questa iterazione (cioè quando la condizione alla riga 3 è vera)
- Supponiamo che $b[1..i-1]$ sia ordinato in modo non decrescente
- Dopo la riga 4 si ha $key = b[i]$

Invariante di ciclo del ciclo interno

L'**invariante di ciclo** del ciclo while interno è:

Ogni volta che viene valutata la condizione alla riga 6:

(I1) $a[1..j] = b[1..j]$ (i primi j elementi non sono stati spostati);

(I2) $a[j+2..i] = b[j+1..i-1]$ (gli elementi $b[j+1], \ldots, b[i-1]$ sono stati spostati di una posizione a destra);

(I3) ogni elemento di $a[j+2..i]$ è strettamente maggiore di key.

Le componenti $a[i+1..n]$ non vengono mai modificate dal ciclo interno.

Il buco e i sotto-array vuoti

- La posizione $j + 1$ è il "**buco**" in cui verrà inserito *key*: il suo contenuto attuale è una copia che verrà sovrascritta
- Un sotto-array $a[l..r]$ con $l > r$ è **vuoto**
- Qualunque proprietà di tutti gli elementi di un sotto-array vuoto è banalmente vera

Esempio

Consideriamo l'array $a = [5, 2, 4, 1, 3]$ all'inizio dell'iterazione $i = 5$ del ciclo esterno: $b[1..5] = [1, 2, 4, 5, 3]$ e $key = 3$. La tabella mostra l'array ogni volta che viene valutata la condizione alla riga 6.

j	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	$a[1..j]$	$a[j+2..5]$	condizione
4	1	2	4	5	3	$[1, 2, 4, 5]$	$[\,]$	$a[4] = 5 > 3$: vera
3	1	2	4	5	5	$[1, 2, 4]$	$[5]$	$a[3] = 4 > 3$: vera
2	1	2	4	4	5	$[1, 2]$	$[4, 5]$	$a[2] = 2 \leq 3$: falsa

A ogni valutazione $a[1..j] = b[1..j]$, $a[j+2..5] = b[j+1..4]$ e tutti gli elementi di $a[j+2..5]$ sono maggiori di 3. Quando il ciclo termina $j = 2$, e la riga 10 pone $a[3] \leftarrow 3$, ottenendo $a[1..5] = [1, 2, 3, 4, 5]$.

Lemma 1: ciclo interno di Insertion Sort

Consideriamo un'iterazione del ciclo esterno di `InsertionSort` *con* $i \in \{2, \ldots, n\}$, *e sia* $b[1..i]$ *il contenuto di* $a[1..i]$ *all'inizio dell'iterazione. Se* $b[1..i-1]$ *è ordinato in modo non decrescente, allora il ciclo while interno termina e, dopo la riga 10, il sotto-array* $a[1..i]$ *contiene gli elementi* $b[1], \ldots, b[i]$ *ordinati in modo non decrescente, mentre* $a[i+1..n]$ *è invariato.*

Dimostrazione (1/5)

Dimostriamo le tre proprietà dell'invariante di ciclo (I1)–(I3).

Inizializzazione.

- Prima della prima valutazione della condizione, $j = i - 1$ e nulla è stato modificato
- Quindi $a[1..i-1] = b[1..i-1]$, cioè (I1) vale
- I sotto-array $a[j+2..i] = a[i+1..i]$ e $b[j+1..i-1] = b[i..i-1]$ sono entrambi vuoti, quindi (I2) e (I3) valgono banalmente

Dimostrazione (2/5)

Conservazione.

- Supponiamo che (I1)–(I3) valgano e che la condizione alla riga 6 sia vera, cioè $j \geq 1$ e $a[j] > key$
- Per (I1), $a[j] = b[j]$, quindi $b[j] > key$
- La riga 7 pone $a[j+1] \leftarrow a[j] = b[j]$; quindi, per (I2),

$$a[j+1..i] = b[j..i-1],$$

  e tutti questi elementi sono maggiori di key (per (I3) e $b[j] > key$)

- Le posizioni $1, \ldots, j$ non vengono modificate

Dimostrazione (3/5)

Conservazione (continua).

- La riga 8 pone $j \leftarrow j - 1$
- Indicando con $j' = j - 1$ il nuovo valore, si ha

$$a[1..j'] = b[1..j'],$$

$$a[j' + 2..i] = a[j + 1..i] = b[j..i - 1] = b[j' + 1..i - 1],$$

  e tutti gli elementi di $a[j' + 2..i]$ sono maggiori di *key*

- Quindi (I1)–(I3) valgono prima della valutazione successiva della condizione

Dimostrazione (4/5)

Terminazione.

- La variabile j parte da $i - 1$ e diminuisce di 1 a ogni iterazione
- Poiché il ciclo si ferma al più tardi quando $j = 0$, il ciclo interno esegue al più $i - 1$ iterazioni e termina
- Quando termina, si ha $j = 0$ oppure $a[j] = b[j] \leq key$. Per (I1)–(I3):
  - $a[1..j] = b[1..j]$ è ordinato e, se $j \geq 1$, il suo elemento più grande $b[j]$ è $\leq key$;
  - $a[j + 2..i] = b[j + 1..i - 1]$ è ordinato e tutti i suoi elementi sono $> key$.

Dimostrazione (5/5)

Terminazione (continua).

- La riga 10 pone $a[j+1] \leftarrow key = b[i]$. Quindi

$$a[1..i] = \big[\, b[1], \ldots, b[j],\ key,\ b[j+1], \ldots, b[i-1] \,\big],$$

  che contiene esattamente gli elementi $b[1], \ldots, b[i]$

- È ordinato in modo non decrescente, perché $b[1..j]$ è ordinato, $b[j] \leq key < b[j+1]$, e $b[j+1..i-1]$ è ordinato

- Le posizioni $i+1, \ldots, n$ non vengono mai accedute

□

Invariante di ciclo del ciclo esterno

- L'**invariante di ciclo** del ciclo while esterno è:

Ogni volta che viene valutata la condizione alla riga 3,
il sotto-array $a[1..i-1]$ è costituito dagli elementi originariamente in $a[1..i-1]$,
ma ordinati in modo non decrescente,
e il sotto-array $a[i..n]$ è uguale al sotto-array originale $a[i..n]$.

Esempio

Consideriamo l'array $a = [5, 2, 4, 1, 3]$ con $n = 5$. La tabella mostra l'array ogni volta che viene valutata la condizione alla riga 3; il sotto-array ordinato $a[1..i-1]$ è evidenziato in blu.

i	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	$a[1..i-1]$ originale	
2	5	2	4	1	3	[5]	
3	2	5	4	1	3	[5, 2]	
4	2	4	5	1	3	[5, 2, 4]	
5	1	2	4	5	3	[5, 2, 4, 1]	
6	1	2	3	4	5	[5, 2, 4, 1, 3]	(il ciclo termina)

Usando l'invariante di ciclo e il lemma sul ciclo interno possiamo dimostrare la correttezza dell'algoritmo `InsertionSort`.

Osservazione 2

Dato un array $a[1..n]$ *di* $n \geq 1$ *numeri naturali, l'algoritmo* `InsertionSort` *termina e, alla fine, l'array* $a[1..n]$ *contiene gli elementi originali riordinati in modo non decrescente:*

$$a[1] \leq a[2] \leq \cdots \leq a[n].$$

Dimostrazione (1/3)

Dimostriamo le tre proprietà dell'invariante di ciclo del ciclo esterno.

Inizializzazione.

- Prima della prima iterazione, $i = 2$ (riga 2) e l'array non è stato modificato
- Il sotto-array $a[1..i-1] = a[1..1]$ è costituito dal singolo elemento originale $a[1]$, ed è banalmente ordinato
- Il sotto-array $a[2..n]$ è quello originale
- Quindi l'invariante vale

Dimostrazione (2/3)

Conservazione.

- Supponiamo che l'invariante valga e che $i \leq n$
- Allora $a[1..i-1]$ contiene gli elementi originali di $a[1..i-1]$ ordinati, e $a[i]$ è l'elemento originale $a[i]$
- Per il Lemma 1, dopo la riga 10 il sotto-array $a[1..i]$ contiene questi i elementi, cioè gli elementi originali di $a[1..i]$, ordinati in modo non decrescente
- e $a[i+1..n]$ è invariato, quindi ancora uguale a quello originale
- La riga 11 pone $i \leftarrow i+1$, quindi l'invariante vale con il nuovo valore di i

Dimostrazione (3/3)

Terminazione.

- La variabile i parte da $2 \leq n+1$ e aumenta di 1 a ogni iterazione, e ogni ciclo interno termina (Lemma 1)
- Quindi il ciclo esterno termina dopo esattamente $n-1$ iterazioni, quando per la prima volta $i = n+1$
- Sostituendo $i = n+1$ nell'invariante di ciclo, il sotto-array $a[1..n]$, cioè l'intero array, è costituito dagli elementi originali ordinati in modo non decrescente

Quindi l'algoritmo è corretto. □

Riepilogo (1/2)

Ciclo interno:

- invariante (I1)–(I3): $a[1..j] = b[1..j]$, $a[j+2..i] = b[j+1..i-1]$, e ogni elemento di $a[j+2..i]$ è $> key$
- la posizione $j+1$ è il "buco" in cui viene inserito key
- esegue al più $i-1$ iterazioni; poi la riga 10 mette key fra $b[j] \leq key$ e $b[j+1] > key$

Il lemma:

- se $b[1..i-1]$ è ordinato, dopo la riga 10 il sotto-array $a[1..i]$ contiene $b[1], \ldots, b[i]$ ordinati, e $a[i+1..n]$ è invariato

Riepilogo (2/2)

Ciclo esterno:

- invariante: $a[1..i-1]$ contiene gli elementi originali di $a[1..i-1]$ ordinati, e $a[i..n]$ è quello originale
- la conservazione segue dal lemma sul ciclo interno
- esattamente $n-1$ iterazioni; con $i = n+1$ l'intero array è ordinato

Cicli annidati:

- un invariante per ciascun ciclo, dimostrando prima quello interno

Correttezza di Binary Search

Introduzione: Correttezza di Binary Search

- In questa sezione dimostriamo la correttezza della ricerca binaria iterativa
- Nei due algoritmi precedenti il ciclo viene sempre eseguito finché la sua condizione diventa falsa
- La ricerca binaria può invece fermarsi in anticipo, con un **return** dentro il ciclo: la dimostrazione deve tenere conto di **entrambe le uscite**

Il problema

Dati un array $a[1..n]$ di n numeri naturali
ordinato in modo non decrescente
e un numero naturale m da cercare,
consideriamo il problema di determinare
se m è uno degli elementi dell'array.

- La ricerca binaria può fermarsi in anticipo, con un **return** dentro il ciclo, e la dimostrazione deve tenere conto di entrambe le uscite
- Questo è lo pseudocodice dell'algoritmo `BinarySearch` (iterativo)

Algorithm: BinarySearch (1/2)

Input : Un array $a[1..n]$ di n numeri naturali ordinato in modo non decrescente e un valore m da cercare

Output : Indice di m in a, oppure -1 se non trovato

```
1 function BinarySearch(a, n, m)
2     left := 1                                   // Estremo sinistro
3     right := n                                  // Estremo destro
4     while left ≤ right do
5         mid := left + ⌊(right − left)/2⌋        // Indice centrale
6         if a[mid] = m then
7             return mid                          // Elemento trovato
8         end
```

Algorithm: BinarySearch (2/2)

```
function BinarySearch(a, n, m) (continua)
    while left ≤ right (continua) do
        if a[mid] < m then
            left ← mid + 1        // Cerca nella metà destra
        end
        else
            right ← mid − 1       // Cerca nella metà sinistra
        end
    end
    return −1                     // Elemento non trovato
end
```

Esegui in Colab: Python · C++ · Java

Invariante di ciclo di BinarySearch

L'**invariante di ciclo** del ciclo while di `BinarySearch` è:

Ogni volta che viene valutata la condizione alla riga 4:

(I1) $1 \leq \mathit{left}$ e $\mathit{right} \leq n$;

(I2) se m è in a, allora m è in $a[\mathit{left}..\mathit{right}]$, cioè

$$m \in \{a[1], \ldots, a[n]\} \implies m \in \{a[\mathit{left}], \ldots, a[\mathit{right}]\}.$$

L'intervallo di ricerca

- A parole: l'algoritmo non scarta mai una porzione dell'array che può contenere m
- Il sotto-array $a[left..right]$ è detto **intervallo di ricerca**
- Contiene $right - left + 1$ elementi, ed è vuoto se $left > right$

Esempio

Ricerca di $m = 10$ in $a = [3, 9, 11, 12, 12, 16, 19]$ ($n = 7$). La tabella mostra l'intervallo di ricerca ogni volta che viene valutata la condizione alla riga 4.

left	*right*	$a[left..right]$	azione
1	7	$[3, 9, 11, 12, 12, 16, 19]$	$mid = 4$, $a[4] = 12 > 10 \Rightarrow right = 3$
1	3	$[3, 9, 11]$	$mid = 2$, $a[2] = 9 < 10 \Rightarrow left = 3$
3	3	$[11]$	$mid = 3$, $a[3] = 11 > 10 \Rightarrow right = 2$
3	2	$[\,]$	$left > right \Rightarrow$ restituisce -1

Poiché 10 non è in a, l'invariante (I2) è banalmente vero; quando l'intervallo diventa vuoto l'algoritmo restituisce correttamente -1.

Osservazione 3

Dati un array $a[1..n]$ di n numeri naturali ordinato in modo non decrescente e un numero naturale m, l'algoritmo `BinarySearch` *termina; restituisce un indice k tale che $a[k] = m$ se m è in a, e restituisce -1 altrimenti.*

Dimostrazione (1/5)

Dimostriamo le tre proprietà dell'invariante di ciclo.

Inizializzazione.

- Le righe 2 e 3 pongono $left = 1$ e $right = n$
- Allora (I1) vale
- e $a[left..right] = a[1..n]$ è l'intero array, quindi (I2) vale

Dimostrazione (2/5)

Conservazione.

- Supponiamo che (I1)–(I2) valgano e che $left \leq right$
- La riga 5 calcola $mid = left + \lfloor (right - left)/2 \rfloor$
- Poiché $0 \leq \lfloor (right - left)/2 \rfloor \leq right - left$, si ha

$$1 \leq left \leq mid \leq right \leq n,$$

  quindi $a[mid]$ è una componente valida dell'array

- Sono possibili tre casi

Dimostrazione (3/5)

Conservazione (continua): i tre casi.

- $a[mid] = m$: l'algoritmo restituisce mid alla riga 7, che è un output corretto.
- $a[mid] < m$: poiché l'array è ordinato, per ogni $k \leq mid$ si ha $a[k] \leq a[mid] < m$. Quindi m non è in $a[left..mid]$, e se m è in $a[left..right]$ allora è in $a[mid + 1..right]$. La riga 10 pone $left \leftarrow mid + 1 \geq 1$, quindi (I1) e (I2) valgono ancora.
- $a[mid] > m$: poiché l'array è ordinato, per ogni $k \geq mid$ si ha $a[k] \geq a[mid] > m$. Quindi, se m è in $a[left..right]$, allora è in $a[left..mid - 1]$. La riga 13 pone $right \leftarrow mid - 1 \leq n$, quindi (I1) e (I2) valgono ancora.

Dimostrazione (4/5)

Terminazione.

- Consideriamo il numero di elementi dell'intervallo di ricerca, $right - left + 1$
- Nel secondo caso il nuovo numero è $right - mid \leq right - left$
- Nel terzo caso è $mid - left \leq right - left$
- In entrambi i casi diminuisce di almeno 1
- Poiché il ciclo continua solo finché $right - left + 1 \geq 1$, il ciclo termina dopo al più n iterazioni

Dimostrazione (5/5)

Terminazione (continua). L'algoritmo finisce in uno di due modi:

- restituisce mid con $a[mid] = m$, che è corretto;
- la condizione $left \leq right$ è falsa, cioè l'intervallo di ricerca $a[left..right]$ è vuoto. Poiché un sotto-array vuoto non contiene m, per (I2) m non è in a, e l'algoritmo restituisce correttamente -1 alla riga 16.

Quindi l'algoritmo è corretto. □

Riepilogo

Invariante di Binary Search:

- (I1) $1 \leq \mathit{left}$ e $\mathit{right} \leq n$
- (I2) se m è in a, allora è nell'intervallo di ricerca $a[\mathit{left}..\mathit{right}]$
- l'algoritmo non scarta mai una porzione dell'array che può contenere m

Due uscite:

- il **return** dentro il ciclo, con $a[\mathit{mid}] = m$
- l'intervallo di ricerca vuoto: per (I2), m non è in a e -1 è corretto
- la dimensione $\mathit{right} - \mathit{left} + 1$ diminuisce di almeno 1: al più n iterazioni

Correttezza dell'algoritmo di Euclide

Introduzione: Correttezza dell'algoritmo di Euclide

- In questa sezione dimostriamo la correttezza dell'algoritmo di Euclide per il massimo comune divisore
- Qui l'invariante non è una proprietà di una porzione di un array, ma un'**uguaglianza fra due numeri** che non cambia mai
- La terminazione si dimostra con il numero naturale m, che diminuisce strettamente

Il problema

Dati due numeri naturali $n, m \geq 1$,
consideriamo il problema di calcolare
il loro massimo comune divisore $\gcd(n, m)$.

- Nell'ultimo esempio l'invariante non è una proprietà di una porzione di un array, ma un'uguaglianza fra due numeri che non cambia mai
- Questo è lo pseudocodice dell'algoritmo di Euclide

Algorithm: EuclideanAlgorithm

Input : Due numeri naturali $n, m \geq 1$
Output : $\gcd(n, m)$

```
1 function EuclideanAlgorithm(n, m)
2     while m ≠ 0 do
3         r := n mod m
4         n ← m
5         m ← r
6     end
7     return n                          // m = 0:  n è il MCD
8 end
```

Esegui in Colab: Python · C++ · Java

Due fatti sul massimo comune divisore

- La dimostrazione si basa sulla seguente proprietà del massimo comune divisore, dimostrata nel capitolo «Algoritmi numerici»:

$$\gcd(n, m) = \gcd(m,\ n \bmod m) \qquad \text{per ogni } n \geq 1,\ m \geq 1.$$

- Usiamo anche la convenzione

$$\gcd(n, 0) = n \qquad \text{per } n \geq 1 :$$

ogni numero naturale divide 0, quindi i divisori comuni di n e 0 sono i divisori di n, e il più grande di essi è n

Invariante di ciclo dell'algoritmo di Euclide

- Indichiamo con n_0 e m_0 i valori di n e m dati in input
- L'**invariante di ciclo** del ciclo while è:

Ogni volta che viene valutata la condizione alla riga 2,
$n \geq 1$, $m \geq 0$, e

$$\gcd(n, m) = \gcd(n_0, m_0).$$

Esempio

Consideriamo $n_0 = 252$ e $m_0 = 105$. La tabella mostra i valori di n e m ogni volta che viene valutata la condizione alla riga 2:

n	m		$\gcd(n, m)$
252	105		21
105	42		21
42	21		21
21	0	21	(il ciclo termina)

Il valore $\gcd(n, m) = 21$ non cambia mai, e quando $m = 0$ l'algoritmo restituisce $n = 21 = \gcd(252, 105)$.

Osservazione 4

Dati due numeri naturali $n_0, m_0 \geq 1$, *l'algoritmo* `EuclideanAlgorithm` *termina e restituisce* $\gcd(n_0, m_0)$.

Dimostrazione (1/3)

Dimostriamo le tre proprietà dell'invariante di ciclo.

Inizializzazione.

- Prima della prima iterazione $n = n_0 \geq 1$ e $m = m_0 \geq 1$
- Quindi banalmente $\gcd(n, m) = \gcd(n_0, m_0)$

Dimostrazione (2/3)

Conservazione.

- Supponiamo che l'invariante valga e che $m \neq 0$, cioè $m \geq 1$
- La riga 3 calcola $r = n \bmod m$, con $0 \leq r < m$
- Le righe 4–5 sostituiscono la coppia (n, m) con (m, r)
- Il nuovo valore di n è $m \geq 1$, il nuovo valore di m è $r \geq 0$, e, poiché $n, m \geq 1$,

$$\gcd(m, r) = \gcd(m,\ n \bmod m) = \gcd(n, m) = \gcd(n_0, m_0)$$

- Quindi l'invariante vale prima dell'iterazione successiva

Dimostrazione (3/3)

Terminazione.

- A ogni iterazione il valore di m viene sostituito da $r < m$, quindi m è un numero naturale che diminuisce strettamente
- Quindi il ciclo termina dopo al più m_0 iterazioni, quando $m = 0$
- Sostituendo $m = 0$ nell'invariante di ciclo otteniamo

$$\gcd(n_0, m_0) = \gcd(n, 0) = n,$$

che è il valore restituito alla riga 7

Quindi l'algoritmo è corretto. □

Due osservazioni

- La dimostrazione non usa l'ipotesi $n_0 \geq m_0$: se $n_0 < m_0$, allora $n_0 \bmod m_0 = n_0$ e la prima iterazione si limita a scambiare i due numeri
- Il limite "al più m_0 iterazioni" basta a dimostrare la terminazione, ma è molto pessimistico:
  - il numero di iterazioni è in realtà $O(\log_2 m_0)$;
  - nell'esempio, 3 iterazioni per $m_0 = 105$.

Riepilogo

Invariante dell'algoritmo di Euclide:

- $n \geq 1$, $m \geq 0$ e $\gcd(n, m) = \gcd(n_0, m_0)$
- conservato grazie alla proprietà $\gcd(n, m) = \gcd(m,\ n \bmod m)$

Terminazione:

- m diminuisce strettamente: al più m_0 iterazioni (in realtà $O(\log_2 m_0)$)
- con $m = 0$ l'invariante fornisce $\gcd(n_0, m_0) = \gcd(n, 0) = n$

Correttezza degli algoritmi ricorsivi

Introduzione: Correttezza degli algoritmi ricorsivi

- Gli invarianti di ciclo sono lo strumento per dimostrare la correttezza degli algoritmi **iterativi**, cioè basati su cicli
- In questa sezione vediamo come si dimostra la correttezza di un algoritmo **ricorsivo**
- Lo strumento è l'induzione matematica, applicata direttamente sulla dimensione dell'istanza

Induzione sulla dimensione dell'istanza

- Un algoritmo **ricorsivo** risolve un'istanza di un problema chiamando se stesso su una o più istanze più piccole
- La sua correttezza si dimostra direttamente per **induzione matematica** sulla dimensione dell'istanza:
  - **caso base**: l'algoritmo restituisce l'output corretto sulle istanze più piccole, che sono risolte senza chiamate ricorsive;
  - **passo induttivo**: sotto l'ipotesi induttiva che le chiamate ricorsive restituiscano l'output corretto sulle istanze più piccole, l'algoritmo ne combina i risultati nell'output corretto per l'istanza corrente.

Terminazione di un algoritmo ricorsivo

- La terminazione segue dal fatto che ogni chiamata ricorsiva è fatta su un'istanza strettamente più piccola
- Quindi prima o poi si raggiunge il caso base

La correttezza degli algoritmi ricorsivi si dimostra per induzione sulla dimensione dell'input.

Questa tecnica viene applicata insieme allo studio degli algoritmi ricorsivi.

Riepilogo

Algoritmi iterativi:

- la correttezza si dimostra con gli invarianti di ciclo

Algoritmi ricorsivi:

- la correttezza si dimostra per induzione sulla dimensione dell'istanza
- caso base: le istanze più piccole, risolte senza chiamate ricorsive
- passo induttivo: si assume che le chiamate ricorsive siano corrette sulle istanze più piccole
- terminazione: ogni chiamata ricorsiva è su un'istanza strettamente più piccola

Riepilogo: punti chiave del capitolo (1/2)

- Un algoritmo è corretto se, per ogni istanza di input, termina e restituisce l'output corretto; i test possono solo mostrare che un algoritmo è sbagliato
- La correttezza di un ciclo si dimostra con un invariante di ciclo: inizializzazione, conservazione e terminazione
- L'invariante, insieme alla negazione della condizione del ciclo, fornisce l'output:
  - $i = n + 1$ per Find Maximum e Insertion Sort;
  - un intervallo di ricerca vuoto per Binary Search;
  - $m = 0$ per l'algoritmo di Euclide.

Riepilogo: punti chiave del capitolo (2/2)

- I cicli annidati richiedono un invariante per ciascun ciclo (Insertion Sort)
- Un **return** dentro il ciclo è una seconda uscita che va dimostrata corretta (Binary Search)
- Gli algoritmi ricorsivi si dimostrano corretti per induzione sulla dimensione dell'input